

Mesh Generation Algorithms: Required Attributes

Advancing Front Method (Advancing Front Mesh Generation Method)

- **Algorithm name:** Advancing Front Method
- **Detail name:** Advancing Front Mesh Generation Method
- **References (BibTeX):**

```
@article{Lo1985,  
  author = {Siu-Hang Lo},  
  title = {A New Mesh Generation Scheme for Arbitrary Planar Domains},  
  journal = {International Journal for Numerical Methods in Engineering},  
  year = {1985},  
  volume = {21},  
  number = {8},  
  pages = {1403--1426},  
  doi = {10.1002/nme.1620210805}  
}
```

- **Publication date:** 1985
- **Resource type:** Journal article (research paper)
- **Era:** 1980s (classic technique)
- **Method paradigm:** Advancing-front, constructive incremental meshing ¹. Elements are added sequentially from the domain boundary inward.
- **Dimension:** Typically 2D (planar domains; extensions to 3D are more complex) ¹.
- **Element type:** Triangles (adds triangular elements to fill the region) ¹.
- **Global rule:** Cover the entire domain by generating new elements that fill space from the boundaries toward the interior. The “front” is the current boundary of unmeshed region ¹.
- **Problem reduction:** Not a decomposition approach – it works directly on the input geometry (no simplification or domain decomposition).
- **Local rule:** At each step, select an active front edge and compute a new vertex (often via geometric heuristics) to form a new triangle. For example, one implementation finds an optimal point on a perpendicular line from the chosen edge and creates a triangle with that new point ².
- **Engine.method:** Iteratively grow the mesh by “advancing” a front of elements. The algorithm maintains a list of boundary segments (the front) and attaches new triangles to these segments until the front vanishes ¹.
- **Engine.objective:** To completely fill the domain with valid elements, ideally of acceptable shape (trying to avoid badly skewed triangles). Quality is enforced heuristically via point placement.
- **Output:** An unstructured triangular mesh covering the domain.
- **Quality metrics:** Seeks reasonably shaped triangles, often measured by minimum angle or edge-length ratios. (No strict threshold guarantee inherent to the method, but implementers may enforce a minimum angle or maximum aspect ratio.)
- **Theoretical guarantees:** The advancing front will terminate (complete coverage) under normal assumptions since each step reduces unmeshed area ³. In fact, a parallel version of advancing

front provably terminates in finitely many steps ³. There is no formal guarantee on minimum angle or global optimality – quality is heuristic.

- **Computational cost:** Typically medium. In 2D it is efficient for moderately large meshes, but must perform geometric searches for valid placement at each step. Complexity is often worse than linear (dependent on front complexity), but usually manageable for classic domains.
- **Practical maturity:** Classic and well-established. Many commercial meshing codes implement an advancing-front option. It has been used since the 1970s/80s (e.g. Lo 1985).
- **Dependencies:** Requires the input domain boundary (possibly segmented) and a function for element size. No complex data prerequisites besides geometry.
- **Complementary suggestions:** Often combined with smoothing or Delaunay refinements afterward. After initial front-based placement, one can improve mesh quality by a global optimization or smoothing pass.
- **Main limitations:** Handling of complex geometry is tricky – fronts may collide or leave holes. Without care, the advancing front can intersect itself and fail. Element size control can be difficult if fronts from opposite sides differ drastically. There is no strong guarantee of element quality (e.g. very small angles can arise).
- **Failure modes:** The front can get “stuck” or infinitely loop if new triangles intersect existing front segments ⁴. Collisions of advancing fronts can produce invalid meshes or poor elements ⁴.
- **Framework conformance:** Conforms to the specified advancing-front paradigm. It uses a global rule (mesh the entire domain from boundary inward) and a local construction rule (attach a triangle to the front edge). It fits the “direct” mesh generation category.
- **Typical deviations:** No notable deviations from the advancing-front framework (it follows the method exactly). It does not perform a domain reduction; it operates on the full domain incrementally.

Bubble Mesh (Sphere-Packing-Based Triangular Meshing)

- **Algorithm name:** Bubble Mesh
- **Detail name:** Bubble Mesh: Sphere-Packing-Based Triangular Mesh Generation
- **References (BibTeX):**

```
@article{Shimada1998,  
  author = {Kenji Shimada},  
  title = {Bubble Mesh: Automated Triangular Meshing of Non-Manifold Geometry by Sphere  
Packing},  
  journal = {Computer-Aided Geometric Design},  
  year = {1998},  
  volume = {15},  
  number = {3},  
  pages = {199--222},  
  doi = {10.1016/S0167-8396(97)00033-X}  
}
```

- **Publication date:** 1998
- **Resource type:** Journal article (research conference/symposium proceedings)
- **Era:** Late 1990s
- **Method paradigm:** Particle/physical simulation approach. It uses a sphere-packing or “bubble” model where mesh vertices are treated as repelling/attracting particles that settle into equilibrium ⁵.

- **Dimension:** 2D, 3D surfaces, and even volumetric domains (“non-manifold geometry”) ⁵ . It can handle planar curves, surfaces, volumes in a unified way.
- **Element type:** Triangles (formed by Delaunay triangulation of the packed points).
- **Global rule:** Fill the domain by densely packing non-overlapping “bubbles” (spheres) of roughly uniform size to represent desired spacing; then compute the Delaunay triangulation of the bubble centers.
- **Problem reduction:** The domain is implicitly represented by allowing bubble centers to fill it under force equilibrium – no further decomposition.
- **Local rule:** Bubbles interact via simple physics: each bubble repels others within a certain range; bubbles on boundaries are placed first. The algorithm gradually adds or removes bubbles to adapt to features. As bubbles settle, the point set adapts to the local feature size.
- **Engine.method:** Simulate an iterative relaxation of bubbles (spheres) under repulsive/attractive forces ⁵ . After convergence of this particle system, compute a Delaunay/Voronoi of the bubble centers to form triangles. A population control scheme ensures bubbles neither overlap nor leave gaps ⁵ .
- **Engine.objective:** Achieve a close-packing of spheres so that the resulting triangulation has nearly uniform element size (matching a sizing function if used) and good shape (near-equilateral triangles). Control of element size is explicit via the bubble spacing ⁶ .
- **Output:** A triangular mesh (often isotropic) covering the domain, automatically conforming to boundaries. The mesh typically has well-shaped triangles.
- **Quality metrics:** Emphasizes isotropy (nearly equilateral triangles) and smooth gradation. Because the bubble centers mimic a centroidal or uniform distribution, the mesh tends to have good element shapes and a uniform quality measure. Precisely, one metric is the node-spacing function control (each bubble enforces a local spacing) ⁶ .
- **Theoretical guarantees:** None formal. This is a heuristic physical method; there is no proof of optimality. However, it is designed to terminate when bubbles reach equilibrium.
- **Computational cost:** High. Simulating many interacting bubbles requires solving many-body forces iteratively, which can be expensive especially in 3D. Complexity grows with number of bubbles and iterations needed to equilibrate.
- **Practical maturity:** Experimental/research. The method was novel when introduced (1998) and produced good results, but it is not as widely used in production as simpler techniques. Still, it influenced later particle-based meshing.
- **Dependencies:** Requires a sizing function or metric to determine bubble sizes, and the ability to compute distances and forces between bubbles. A CAD geometry or distance field defines where bubbles can exist.
- **Complementary suggestions:** After initial bubble packing, one may use a Delaunay refinement (e.g. Chew or Ruppert) to further improve mesh quality or handle any irregularities. This can fix any small-angle triangles if they occur.
- **Main limitations:** Very complex to implement and tune. Controlling the bubble interaction parameters (force law, sizing) can be tricky. It may be slow for large domains. It also tends to require careful initialization (bubbles by dimension order) to succeed.
- **Failure modes:** If bubbles are initialized poorly or forces are unbalanced, the bubble system may oscillate or converge to a poor configuration (leading to uneven mesh). Large variations in feature size can cause packing difficulties.
- **Framework conformance:** It deviates from the classical advancing-front or Delaunay-refinement frameworks. It is a physical/optimization approach rather than a global rule/partition. In the schema, it can be seen as building a point set (global rule) and then output via Delaunay, rather than stepwise front or refinement.
- **Typical deviations:** The algorithm does not explicitly follow the “global rule / local rule” format. It uses continuous forces, so it falls outside simple local-rule categories. This is acceptable because it is fundamentally a different paradigm (particle relaxation) for mesh generation.

Chew’ s Refinement (Delaunay Refinement for Quality Triangulation)

- **Algorithm name:** Chew’ s Refinement (Chew’ s Delaunay Refinement)
- **Detail name:** Chew’ s Delaunay Refinement Algorithm for Quality Triangulation
- **References (BibTeX):**

```
@article{Chew1989,  
  author = {L. Paul Chew},  
  title = {Constrained Delaunay Triangulations},  
  journal = {Algorithmica},  
  year = {1989},  
  volume = {4},  
  pages = {97--108},  
  doi = {10.1007/BF01553881}  
}  
  
@inproceedings{ChewSecondAlgorithm,  
  author = {L. Paul Chew},  
  title = {Guaranteed-Quality Mesh Generation for Curved Surfaces},  
  booktitle = {Proceedings of the Ninth Annual Symposium on Computational Geometry},  
  year = {1993},  
  pages = {274--280}  
}  
  
@techreport{Shewchuk1997,  
  author = {Jonathan Richard Shewchuk},  
  title = {Delaunay Refinement Algorithms for Triangular Mesh Generation},  
  institution = {Carnegie Mellon University},  
  year = {1997}  
}
```

- **Publication date:** 1989 (and extended results in 1993, 1997)
- **Resource type:** Journal article and conference papers
- **Era:** Late 1980s / early 1990s (theory work in computational geometry)
- **Method paradigm:** Delaunay refinement (Steiner point insertion). The mesh is refined by repeatedly inserting new vertices (Steiner points) to eliminate bad-quality triangles in a Delaunay triangulation.
- **Dimension:** 2D triangulation (surfaces embedded in 3D can be handled by constrained Delaunay).
- **Element type:** Triangles.
- **Global rule:** The output must be a quality-conforming Delaunay triangulation of the input geometry. The global quality requirement is usually a minimum angle bound.
- **Problem reduction:** Begin with a Delaunay (or constrained Delaunay) triangulation of the domain vertices. Then refine by adding points. No major problem reduction beyond initial Delaunay.
- **Local rule:** Identify a “poor-quality” triangle (e.g. with a small angle). Compute its circumcenter. If that center is safe (does not encroach a boundary edge), insert it; otherwise split the offending boundary segment by inserting its midpoint. Each insertion affects only local triangles (forms a cavity) which is retriangulated.
- **Engine.method:** Iterative point insertion. In each step, perform a Delaunay update: delete all triangles whose circumcircles contain the new point, then re-triangulate the resulting polygonal hole. This is essentially Bowyer–Watson insertion.

- **Engine.objective:** Improve element quality by eliminating triangles that violate the quality criterion. The algorithm aims to enforce a minimum angle (or maximum circumradius/shortest-edge ratio).
- **Output:** A (constrained) Delaunay triangulation that satisfies the quality constraint. The mesh has no angles smaller than a guaranteed bound (modulo boundary issues).
- **Quality metrics:** Minimum angle (Chew's bound is about 28.6°) ⁷. Also graded with respect to the local feature size (mesh size adapts to geometry).
- **Theoretical guarantees:** Termination is guaranteed (it won't insert infinitely many points) as long as the domain is non-acute and the angle threshold is below a critical value. Chew proved a bound: the minimum angle is at least $\sim 28.6^\circ$ ⁷. Later refinements (Shewchuk) relaxed conditions.
- **Computational cost:** Medium-high. Each insertion requires a local triangulation update (often log-time with proper data structures), but the number of insertions can grow if the threshold is high. Overall often near linear or $n \log n$, but depends on output size.
- **Practical maturity:** Mature and widely used. Chew's algorithm forms the basis of many meshers (e.g. Triangle by Shewchuk). It is considered classic.
- **Dependencies:** An initial Delaunay triangulation and a list of bad triangles. Requires ability to compute circumcenters and check segment encroachment.
- **Complementary suggestions:** Usually combined with boundary recovery and smoothing. Many implementations (like Triangle) also post-process to improve angles or add a few more points to handle acute inputs.
- **Main limitations:** Restricted to domains without tiny input angles (Chew's analysis assumes no very small angles in the input, though practical versions handle most cases). Chew's original bound (28.6°) is slightly pessimistic; Ruppert's later analysis allowed $\sim 20.7^\circ$ threshold with less restriction.
- **Failure modes:** If input has extremely small angles, the algorithm may need to insert many points or fail (violating size constraints near corners). Without modifications, it may not terminate if input angles are too sharp.
- **Framework conformance:** Conforms to Delaunay refinement framework. Global rule is "maintain Delaunay", local rule is "insert circumcenter of bad triangle (or split encroached segment)". It follows the schema precisely.
- **Typical deviations:** Some implementations relax the quality threshold adaptively near small input angles, deviating slightly to ensure convergence (allowed because of feature-size constraints). This is a controlled deviation that fits practical use.

CVT / Lloyd Optimization (Centroidal Voronoi Tessellation)

- **Algorithm name:** CVT / Lloyd Optimization
- **Detail name:** Centroidal Voronoi Tessellation (CVT) / Lloyd's Algorithm
- **References (BibTeX):**

```
@article{Du1999,
  author = {Qiang Du and Vance Faber and Max Gunzburger},
  title = {Centroidal Voronoi Tessellations: Applications and Algorithms},
  journal = {SIAM Review},
  year = {1999},
  volume = {41},
  number = {4},
  pages = {637--676},
```

```
doi = {10.1137/S0036144599352836}
}
```

- **Publication date:** 1999
- **Resource type:** Journal review article
- **Era:** Late 1990s (SIAM Review)
- **Method paradigm:** Optimization / iterative relaxation. Lloyd' s algorithm is used to find a centroidal Voronoi tessellation – points move to cell centroids repeatedly ⁸ .
- **Dimension:** Typically 2D (planar regions or surfaces). Also extendable to 3D with centroidal Voronoi.
- **Element type:** Triangles (or polygonal cells). A CVT is often triangulated via its dual Delaunay, so output is a triangular mesh (often generated by Delaunay of the optimized points). Sometimes used to generate quads via combining dual edges.
- **Global rule:** Points should coincide with centroids (centers of mass) of their Voronoi cells; globally this minimizes a certain energy (mean-squared distance). All cells are well-shaped.
- **Problem reduction:** None in the geometric sense – the domain stays the same. It reduces to an unconstrained optimization problem of point positions.
- **Local rule:** In Lloyd' s iteration, for each site, compute its Voronoi cell and move the site to the centroid of that cell ⁸ . Then recompute Voronoi.
- **Engine.method:** Repeated Voronoi diagram computation and centroid averaging. At each iteration: (1) compute Voronoi of current sites; (2) integrate each cell to find its centroid; (3) move each site to its centroid ⁹ . Repeat until convergence.
- **Engine.objective:** Minimize the integral of squared distances from arbitrary points in the domain to the nearest site. Equivalently, maximize uniformity and isotropy of cells.
- **Output:** A Voronoi diagram (and its dual Delaunay triangulation). The triangulation is such that each triangle edge is typically not too skinny. It yields a mesh of convex cells (Voronoi) or triangles (Delaunay) that are uniform in size.
- **Quality metrics:** Cells are convex and well-shaped; triangles (if using Delaunay) have high minimum angles (Delaunay tends to avoid skinny triangles). Lloyd' s CVT maximizes the minimum angle among triangulations and tends to equalize edge lengths. Uniformity (all edges similar length) is a measure.
- **Theoretical guarantees:** Lloyd' s method converges to a local optimum of the CVT energy. It is guaranteed to reduce the energy each iteration, but may only find a local minimum, not global ⁸ . It terminates when point moves are below a threshold.
- **Computational cost:** Medium-high. Each iteration requires computing a Voronoi diagram (cost $\sim O(n \log n)$) and centroids. Many iterations may be needed for convergence. Overall cost depends on number of points and iterations.
- **Practical maturity:** Well-established. CVT and Lloyd' s algorithm are standard in mesh generation and remeshing (often for isotropic meshes). There are robust implementations.
- **Dependencies:** Requires a way to compute Voronoi cells (or approximations), and an integration scheme for centroids. Also a density or sizing function can be incorporated to produce graded meshes.
- **Complementary suggestions:** Often initialized with a coarse grid or random points. One may combine CVT with anisotropic metrics (e.g. by working in a transformed space or using power diagrams). The resulting points are usually retriangulated by Delaunay.
- **Main limitations:** Lloyd' s algorithm is purely isotropic by default; anisotropy or directional features are not captured unless explicitly added (e.g. metric tensor). Convergence can stall in symmetrical local minima.
- **Failure modes:** If density varies strongly, or domain geometry is complex, Lloyd' s iteration might cluster points incorrectly or converge slowly.

- **Framework conformance:** Does not follow a front or greedy rule. It is a global iterative optimization. In the framework terms, the “engine” is the iterative repositioning of points according to Voronoi centroids. This deviates from the local/global rule format, since Lloyd’s is a global relaxer.
- **Typical deviations:** It does not produce a directly quality-guaranteed mesh, so one often applies a Delaunay triangulation after convergence. This step is consistent with outputting a mesh, but the core Lloyd algorithm itself is more like an optimization loop than a constructive meshing rule.

Delaunay Triangulation (Bowyer–Watson)

- **Algorithm name:** Bowyer–Watson Delaunay Triangulation
- **Detail name:** Bowyer–Watson Incremental Delaunay Triangulation Algorithm
- **References (BibTeX):**

```
@article{Bowyer1981,
  author = {Adrian Bowyer},
  title = {Computing Dirichlet Tessellations},
  journal = {The Computer Journal},
  year = {1981},
  volume = {24},
  number = {2},
  pages = {162--166},
  doi = {10.1093/comjnl/24.2.162}
}
```

- **Publication date:** 1981
- **Resource type:** Journal article (computational geometry)
- **Era:** Early 1980s (a classic computational geometry result)
- **Method paradigm:** Incremental Delaunay construction (computational geometry). The method builds a Delaunay triangulation by adding points one at a time.
- **Dimension:** Arbitrary dimensions (Bowyer–Watson works in 2D, 3D, etc.) ¹⁰. Most often used in 2D meshing.
- **Element type:** Triangles (in 2D). In general dimension, simplices.
- **Global rule:** Output is the Delaunay triangulation of the point set. The Delaunay property (no point inside any triangle’s circumcircle) is the global criterion.
- **Problem reduction:** No reduction; it’s direct: given all points, form the triangulation.
- **Local rule:** When inserting a new point, locate all existing triangles whose circumcircles contain the point (“bad triangles”). Remove those triangles (they form a cavity). Then triangulate the cavity by connecting the new point to the boundary of the hole ¹⁰. This local re-triangulation restores Delaunay property.
- **Engine.method:** Iteratively insert points. Start with a large “super-triangle” covering all points. For each new point, find and remove affected triangles, re-triangulate. This ensures after each insertion the triangulation remains Delaunay.
- **Engine.objective:** Achieve a valid Delaunay triangulation (maximizing minimum angle property globally) efficiently.
- **Output:** A Delaunay triangulation (or Constrained Delaunay if segments are constrained) of all input points. The dual of this is the Voronoi diagram.
- **Quality metrics:** Delaunay triangulation maximizes the minimum angle among all possible triangulations of the same points (so it tends to avoid skinny triangles). Edge-length distribution is relatively uniform.

- **Theoretical guarantees:** It produces the correct Delaunay triangulation by construction. The expected time complexity is $O(N \log N)$ if efficient point location is used ¹⁰. In worst-case (pathological point order), it can degrade to $O(N^2)$ ¹⁰.
- **Computational cost:** Low to medium. With good data structures, average case is near $O(N \log N)$ ¹⁰. It is quite fast and robust for typical point sets.
- **Practical maturity:** Classic and ubiquitous. Bowyer–Watson is a standard algorithm implemented in most mesh libraries.
- **Dependencies:** Requires the full set of points and any constrained segments. Needs geometric routines (circumcircle tests, point-location).
- **Complementary suggestions:** Often used as a subroutine in refinement or other generators. For example, many methods maintain a Delaunay triangulation while modifying point set (like Chew's and Ruppert's algorithms).
- **Main limitations:** In isolation, it does not enforce any mesh sizing or quality beyond Delaunay. It requires knowing all points in advance (though it can build incrementally). Handling fixed boundaries requires a constrained-Delaunay variant.
- **Failure modes:** Rare for standard Delaunay (only degeneracies like co-circular points cause technical issues, handled by simulation of simplicity). Computationally, the worst-case quadratic time can happen if points are added in bad order (though rare with random insertion).
- **Framework conformance:** It is not a “mesh generator” per se, but a triangulation algorithm. In framework terms, it fits the problem reduction approach: no division, but it does “re-triangulate” locally. It serves often as the core engine for Delaunay-based generators.

DistMesh (Distance-Field-Based Spring Relaxation Mesh Generator)

- **Algorithm name:** DistMesh
- **Detail name:** DistMesh: Distance-Field-Based Spring Relaxation Mesh Generator
- **References (BibTeX):**

```
@article{Persson2004,
  author = {Per-Olof Persson and Gilbert Strang},
  title = {A Simple Mesh Generator in MATLAB},
  journal = {SIAM Review},
  year = {2004},
  volume = {46},
  number = {2},
  pages = {329--345},
  doi = {10.1137/S0036144503429121}
}
```

- **Publication date:** 2004
- **Resource type:** Journal article (numerical analysis / meshing)
- **Era:** Early 2000s
- **Method paradigm:** Iterative relaxation using a truss/spring analogy. The mesh edges are treated as springs pulling points to equilibrium ¹¹. The geometry is defined by a signed distance function.
- **Dimension:** Primarily 2D (originally implemented for planar mesh generation). Extended versions exist for 3D tetrahedral meshing.
- **Element type:** Triangles (in 2D; DistMesh also supports tetrahedra in 3D with some modifications).

- **Global rule:** Nodes should follow the zero level-set of the distance function (boundary) and the edge lengths should match a user-defined size function. The goal is a quasi-uniform mesh respecting a sizing field.
- **Problem reduction:** The domain is specified implicitly (distance function); DistMesh does not subdivide domain but moves points continuously.
- **Local rule:** At each iteration, apply a force on each node from its connected edges (springs) to drive edges to a desired length. Then update node positions by a small step of spring forces, project boundary nodes back onto the boundary, and re-triangulate via Delaunay. Repeat until equilibrium ¹¹.
- **Engine.method:** Repeatedly solve for equilibrium of a linear spring system on a triangular mesh. Specifically, mesh vertices are connected by “bars” (edges) with a target length. One solves (via a simple iteration) for node positions that balance spring forces ¹¹, then updates the triangulation with Delaunay.
- **Engine.objective:** Equalize edge lengths to a prescribed size function across the mesh. The algorithm enforces that each edge length is as close as possible to the target (making an “isotropic” mesh) ¹².
- **Output:** A Delaunay triangulation of the node set that has approximately uniform edge lengths (as given by the distance-based sizing). The mesh is typically high-quality (near equilateral triangles) due to uniform edge lengths.
- **Quality metrics:** Near-equal edge lengths imply good minimum angle. The paper reports “surprisingly high quality” meshes ¹³. No explicit angle bounds, but in practice skew is low.
- **Theoretical guarantees:** No formal mesh-quality guarantee is proven. The method is heuristic but tends to converge. It will terminate when movement is small.
- **Computational cost:** Medium. Each iteration requires solving a sparse linear system or similar for equilibrium (often via Jacobi or Gauss-Seidel) and a Delaunay retriangulation. Convergence can take many iterations, especially for large meshes.
- **Practical maturity:** Well-known in academic and some practical contexts. The code is simple (MATLAB implementation available) and is used in research. It is not as ubiquitous as some industrial meshers, but widely respected for ease of use.
- **Dependencies:** Requires a signed distance function (or any implicit description) of the geometry to project points onto boundaries. Also requires a target edge-length function. Uses a Delaunay library for triangulation (e.g. Triangle).
- **Complementary suggestions:** One can apply a smoothing pass after convergence if needed. Since output is already fairly uniform, often little additional optimization is needed. For domains with complex boundaries, combining with a boundary-fitting step can help.
- **Main limitations:** It is heuristic and may need tuning (choice of step size in relaxation, target lengths). Convergence can stall if steps are too large. It may also have difficulty with extremely small or anisotropic features, since it is inherently isotropic.
- **Failure modes:** If the distance function is inaccurate (points erroneously move off boundary) or if the iteration overshoots, mesh tangling or invalid triangles can occur. Poor initial placement can also lead to slow convergence.
- **Framework conformance:** DistMesh is essentially an optimizer. It uses a global sizing rule and local spring equilibrium. This is somewhat different from the classic “global vs local rule” framework, but one can interpret the distance function as a global size rule and the spring relaxation as the local rule. Thus it partially conforms, though it is more of a continuous solver than a discrete step-by-step generator.
- **Typical deviations:** It does not follow a fixed mesh-generation rule but rather evolves the mesh gradually. This can be seen as a deviation from “constructive” methods. However, it fits an extended view of the framework as an “iterative local optimization” engine.

FreeMesh-RL (Reinforcement Learning for Quad Meshes)

- **Algorithm name:** FreeMesh-RL
- **Detail name:** FreeMesh-RL: SAC-Based Reinforcement Learning for Automatic Quadrilateral Mesh Generation
- **References (BibTeX):**

```
@article{Pan2022FreeMeshRL,  
  author = {Jie Pan and Jingwei Huang and Gengdong Cheng and Yong Zeng},  
  title = {Reinforcement Learning for Automatic Quadrilateral Mesh Generation: A Soft Actor-Critic Approach},  
  journal= {Neural Networks},  
  year = {2022},  
  note = {Preprint submitted to Neural Networks, arXiv:2203.11203}  
}
```

- **Publication date:** 2022 (preprint)
- **Resource type:** Preprint (arXiv) / journal submission
- **Era:** 2020s (very recent)
- **Method paradigm:** Reinforcement Learning (AI). The algorithm learns a policy to generate meshes by treating mesh construction as a sequential decision process with states and actions. It uses the Soft Actor-Critic (SAC) RL technique.
- **Dimension:** 2D only (the current implementation is limited to planar domains) ¹⁴.
- **Element type:** Quadrilaterals (focus is all-quad mesh generation).
- **Global rule:** The trained policy seeks to produce a high-quality quad mesh for the entire domain; effectively it learns when and where to place or combine elements. There is no hard-coded global rule, but the objective function (via rewards) encourages good coverage and quality.
- **Problem reduction:** It works directly on the full domain. Internally, it may decompose tasks (e.g. refine regions) but this is learned, not explicitly defined.
- **Local rule:** The policy performs local mesh operations (e.g. meshing a local region or adding quads) based on the current partial mesh state. These local decisions are guided by the learned model rather than predefined heuristics.
- **Engine.method:** A neural-network policy is trained via reinforcement learning (specifically SAC). The training loop simulates many mesh generation episodes, optimizing a reward that encodes mesh quality ¹⁵. At inference, the trained policy generates a mesh in one pass.
- **Engine.objective:** Maximize a quality-based reward (likely related to element quality metrics, though exact formulation depends on implementation). The goal is to produce meshes comparable to commercial systems without human intervention.
- **Output:** A fully automated quadrilateral mesh for the input domain. The result aims to match or approach the quality of expert-generated meshes.
- **Quality metrics:** Empirically, the method targets standard quad quality measures (e.g. element shape regularity, alignment). The paper notes performance “comparable quality to commercial systems” ¹⁵. The reward likely includes metrics like angle deviation or aspect ratio.
- **Theoretical guarantees:** None. Being ML-based, it has no formal mesh guarantees; results depend on training quality and neural network.
- **Computational cost:** Very high during training (RL training is expensive). At generation time, inference cost is moderate (the learned policy runs in one sequence). The paper notes it can generate meshes automatically after training.

- **Practical maturity:** Experimental. This is a novel research prototype. While promising, it is not yet an engineering-ready solution for all cases. The authors themselves note current limitation to 2D ¹⁴.
- **Dependencies:** Requires a trained RL model (complex ML framework), as well as the environment definition. Domain geometry is the input. Depends heavily on the definition of state, action space, and reward.
- **Complementary suggestions:** As an emerging approach, it can be complemented by classical methods in a hybrid fashion. For instance, one might use RL for coarse quad placement and then refine or smooth with traditional algorithms.
- **Main limitations:** Limited to 2D domains so far ¹⁴. Training is expensive and the outcome can be unpredictable. It may not generalize well beyond the kinds of geometries seen during training.
- **Failure modes:** The learned policy might produce invalid meshes (holes or overlaps) if not properly trained. It can also “forget” to mesh certain regions. Robustness depends on training coverage.
- **Framework conformance:** It does not fit neatly into the classical framework (global rules/local rules) because the rules are learned. In effect, it is a black-box engine. We can say Engine.method = learned mesh policy, which is not a standard algorithm. It deviates by replacing all rule definitions with a learned model.
- **Typical deviations:** Entirely departs from explicit rules. This deviation is by design: it is an instance of a new paradigm (machine learning) not captured by traditional meshing frameworks.

Instant Meshes (Field-Aligned Quadrilateral Meshing)

- **Algorithm name:** Instant Meshes
- **Detail name:** Instant Meshes: Instant Field-Aligned Quadrilateral Meshing
- **References (BibTeX):**

```
@inproceedings{Jakob2015,
  author = {Wenzel Jakob and Marco Tarini and Daniele Panozzo and Olga Sorkine-Hornung},
  title = {Instant Field-Aligned Meshes},
  booktitle = {ACM Transactions on Graphics (SIGGRAPH Asia 2015)},
  year = {2015},
  volume = {34},
  number = {6},
  pages = {189:1--189:15},
  doi = {10.1145/2816795.2818078}
}
```

- **Publication date:** 2015
- **Resource type:** Conference proceedings (SIGGRAPH Asia)
- **Era:** 2010s (modern research algorithm)
- **Method paradigm:** Local geometric smoothing guided by a cross field. It treats mesh vertices with a smoothing operator that aligns edge orientations to a smooth directional field. There is no global optimization; all operations are local and parallelizable.
- **Dimension:** 2D manifold surfaces (the algorithm remeshes 3D surface models into 2D meshes). It can also remesh planar regions.
- **Element type:** Primarily quadrilateral (quad-dominant). It can also produce triangular meshes if desired, but main output is an all-quad or near-all-quad mesh aligned to features.
- **Global rule:** Produce an isotropic quad mesh that aligns to a low-distortion field. Edges should align to a smoothly varying 4-way (symmetric) direction field on the surface.

- **Problem reduction:** The method works on the full mesh. The input surface may be parameterized or left as is; the algorithm does not subdivide the domain.
- **Local rule:** Iteratively apply a local optimization at each vertex that moves vertices and re-connects edges to better align with the target orientation. This involves solving a small local linear system to update positions and connectivity. The orientation of edges is adjusted to follow the smooth field.
- **Engine.method:** Repeated local smoothing steps. Specifically, the algorithm constructs a “position field” or orientation function on each face and then alternates solving for vertex positions that minimize an energy, with combinatorial changes in connectivity to enforce 4-way symmetry ¹⁶. All operations (position update and edge swap) are done locally.
- **Engine.objective:** Optimize edge orientations and vertex positions to achieve an isotropic mesh (edges of uniform length) with edges aligned to surface features or a user-defined field ¹⁶. Sharp features are preserved by snapping edges to them.
- **Output:** A high-quality quadrilateral mesh. The examples show meshes with nearly regular quads that follow geometry. It is nearly all-quad, with a small number of triangles only if needed.
- **Quality metrics:** Very high isotropy (most quads are near squares). Sharp features alignment is also a measure (edges snap to feature lines) ¹⁷. The method aims to maximize the angle quality (close to 90°) and smoothness of quad size.
- **Theoretical guarantees:** None formal. Convergence is typically fast in practice. The algorithm is non-linear, but each step reduces a local error. It avoids a global solve to remain simple.
- **Computational cost:** Low to medium. Each iteration does local computations that scale linearly with mesh size, and no global linear system is solved ¹⁸. The authors report it executes “instantly” for meshes with up to hundreds of thousands of elements ¹⁸. Very large meshes (millions of faces) can still be handled in seconds to minutes.
- **Practical maturity:** High for surface remeshing. The method has open-source implementations (IGL/InstantMesh), and has been integrated into commercial tools (e.g. Modo) ¹⁹. It is considered engineering-ready for quad meshing tasks.
- **Dependencies:** Requires a cross-field or alignment field on the surface (the algorithm computes this internally). Uses local geometry (triangulated surface). Does not depend on any external optimization package.
- **Complementary suggestions:** Often preceded by a field design step (to set directional constraints). The output can be further smoothed or optimized if extreme precision is required. Also can be combined with simplification or refinement for very coarse/fine target sizes.
- **Main limitations:** Primarily works on surfaces; 3D volumetric meshing with quads/hex is not addressed. The algorithm produces a “quad-dominant” mesh, meaning a few triangles may remain (though typically very few). Handling of non-manifold or extremely noisy geometry can be tricky.
- **Failure modes:** If the cross field has singularities (inevitable in some topologies), the algorithm will create irregular vertices with valence $\neq 4$. These are acceptable but indicate a mismatch in field design. Very poor initial connectivity or extremely skewed inputs might cause suboptimal results.
- **Framework conformance:** It fits loosely: the “global rule” is implicit (align quads to field), and the “local rule” is the smoothing operator. The engine is local optimization. It deviates from classical meshing: there is no explicit element creation step, but rather continuous adjustment of vertex positions and connectivity.
- **Typical deviations:** It avoids any explicit global data structure (like a front or tree). Instead, it does entirely local moves. This means it does not strictly follow a partition/slicing approach. But it does conform to the broad idea of a mesh generator that optimizes element quality.

MIQ (Mixed-Integer Quadrangulation)

- **Algorithm name:** MIQ
- **Detail name:** Mixed-Integer Quadrangulation (MIQ)
- **References (BibTeX):**

```
@article{Bommes2009,  
  author = {David Bommes and Henrik Zimmer and Leif Kobbelt},  
  title = {Mixed-Integer Quadrangulation},  
  journal = {ACM Transactions on Graphics},  
  year = {2009},  
  volume = {28},  
  number = {3},  
  pages = {77:1--77:10},  
  doi = {10.1145/1531326.1531383}  
}
```

- **Publication date:** 2009
- **Resource type:** Journal article (SIGGRAPH paper)
- **Era:** Late 2000s
- **Method paradigm:** Global optimization with field and parameterization. MIQ formulates the quadrangulation problem as two mixed-integer optimization steps: first compute an optimal cross field (smooth symmetric direction field with constraints), then compute a seamless global parameterization following that field ²⁰.
- **Dimension:** 2D manifold surfaces (triangle meshes).
- **Element type:** Quadrilaterals (all-quad mesh on the surface).
- **Global rule:** Minimize distortion of a quad layout: the user may specify a few feature directions (sparse constraints), and the algorithm automatically determines the number and placement of irregular vertices (singularities) to align to geometry ²⁰. The mesh strives to follow principal curvature or feature directions.
- **Problem reduction:** The input is first cut into a topological disk (parametrization cuts). This is part of the method to allow a global parametrization. After cutting, the mesh is flattened via parameterization.
- **Local rule:** There is no simple local generation rule. Instead, constraints are global: the angles of a cross field at each face and the jump rotation (integer multiples of 90°) across edges are determined by solving a linear system with integer constraints ²⁰. This yields a seamless map.
- **Engine.method:** Solve two sparse mixed-integer optimization problems: (1) find a 4-way rotationally symmetric direction (cross) field that is as smooth as possible and satisfies any specified alignments; (2) compute a seamless parametrization whose iso-lines follow that cross field. Both steps reduce to solving Poisson-like equations plus enforcing integer rotation constraints ²⁰. The integer variables encode how mesh parameterization aligns to the cross field. The paper uses a specialized solver (graph-cuts or integer solver).
- **Engine.objective:** Minimize global distortion of the quad layout and satisfy alignment to the directional field. The formulation seeks an “as smooth as possible” field and parameterization. The ultimate goal is a quad mesh that is aligned to important directions and as regular as possible.
- **Output:** An all-quadrilateral mesh of the surface. The mesh has exactly the right singular vertices (valence $\neq 4$) determined by the integer solver. The quads are aligned to the computed field.
- **Quality metrics:** Quads are optimized for alignment and fairness: edges follow the field, which often corresponds to curvature directions, yielding aligned rows of quads. The algorithm balances

quad shape quality (close to squares) and alignment. The output shown in examples has very smooth, regular quads.

- **Theoretical guarantees:** No formal guarantees beyond achieving a local minimum of the formulated optimization. The method is deterministic and will converge to an optimal (mixed-integer) solution given the solver's convergence.
- **Computational cost:** High. Solving mixed-integer systems (especially the integer labeling of edges) is expensive. The authors use an efficient solver, but runtimes are significant for large meshes. This is not real-time; it is an offline algorithm.
- **Practical maturity:** Advanced research prototype. It produced high-quality meshes and inspired many subsequent works. An implementation exists (the authors' code), but it is more academic. It's used when the best quality is needed, not for quick meshing.
- **Dependencies:** Requires a triangle mesh of the surface, a solver for mixed-integer linear problems (the authors released a solver for their use), and optionally feature constraints (some boundary or symmetry directions).
- **Complementary suggestions:** Can be combined with mesh simplification before processing to reduce mesh size. Also, if purely quad meshes are needed, it can serve as a final pass after a coarse quad placement.
- **Main limitations:** Complexity and runtime. Handling very large meshes (millions of faces) is challenging. Also, it produces exactly all quads, which may not be needed if some triangles are acceptable. It also requires the user to provide or choose directional constraints for best results.
- **Failure modes:** If the cross-field solver fails or picks a poor solution, the resulting mesh can have irregular singularities or distortions. Also, incorrect mesh cuts (topology) can cause artifacts. In practice, the method is robust given careful implementation.
- **Framework conformance:** The method is largely outside the usual global/local rule framework. It is a global parameterization method. One can say `Engine.method = mixed-integer solver` and `Engine.objective = minimal distortion`, which is unlike stepwise meshing. It "deviates" from discrete rule frameworks by performing continuous optimization in a high-dimensional space.

Paving (All-Quadrilateral Advancing-Front)

- **Algorithm name:** Paving
- **Detail name:** Paving: All-Quadrilateral Advancing-Front Mesh Generation
- **References (BibTeX):**

```
@article{Blacker1991,  
  author = {Ted D. Blacker and Michael B. Stephenson},  
  title = {Paving: A New Approach to Automated Quadrilateral Mesh Generation},  
  journal = {International Journal for Numerical Methods in Engineering},  
  year = {1991},  
  volume = {32},  
  number = {4},  
  pages = {811--847},  
  doi = {10.1002/NME.1620320410}  
}
```

- **Publication date:** 1991
- **Resource type:** Journal article (mesh generation)
- **Era:** Early 1990s
- **Method paradigm:** Advancing-front (direct). Paving places rows of quads sequentially from the boundary inward, very much like its 3D hex analog (plastering) ²¹.

- **Dimension:** 2D planar domains or surface parameterization.
- **Element type:** Quadrilaterals (all-quad output).
- **Global rule:** Fill the entire domain with quadrilaterals such that edges of quads align in strips parallel to the boundary. Each new row of quads is attached to the existing front.
- **Problem reduction:** No explicit reduction; operates on full domain.
- **Local rule:** At each step, extend the current front by creating a whole strip (row) of quadrilaterals. The strip follows the shape of the front geometry, with new nodes placed to allow a consistent quad row. Special “seam” operations are used to handle front interactions.
- **Engine.method:** Iteratively build “layers” of quads. Paving maintains a set of front segments. It then creates a new layer of quads by sweeping along the front. The position of new nodes is often chosen to maintain element shape and alignment. When two advancing fronts meet, a seam or gap may be closed by special steps.
- **Engine.objective:** Achieve an all-quad mesh with smooth gradation from the boundary inward. The algorithm tries to match quad sizes so that opposing fronts do not collide too abruptly.
- **Output:** An all-quadrilateral mesh covering the domain. Quads tend to form well-aligned “rows” that parallel domain edges.
- **Quality metrics:** Quads are aimed to have good aspect ratios (near squares). The method naturally aligns quads to boundaries, which improves boundary-conformity.
- **Theoretical guarantees:** No formal guarantee of success on arbitrary shapes. In practice, as long as input boundaries have an even number of segments, an all-quad mesh can be generated ²². However, theory is largely experimental.
- **Computational cost:** Moderate. Each “row” generation is linear in the current front size. The algorithm is essentially deterministic and fast for typical problems.
- **Practical maturity:** Well-known and implemented in many mesh tools. Paving (and its variants) became a standard method for quadrilateral meshing after its introduction.
- **Dependencies:** Needs the domain boundary (partitioned into edges with an even count). If boundaries have sharp features, these guide front placement.
- **Complementary suggestions:** Often followed by smoothing or further topological fixes (like Q-Morph’s clean-up) to handle irregular nodes. White and Kinney’s work (2004) also suggests element-by-element improvements on top of paving.
- **Main limitations:** Paving can struggle if opposing fronts have very different spacing or curvature; this can create overlap or gaps ⁴. If the number of boundary edges is odd, a purely quad mesh is impossible (requires a triangle). Also, sharply non-orthogonal boundaries can cause skewed quads.
- **Failure modes:** Front intersections (“over-meshing”) if quads from two sides collide ⁴. If the algorithm does not detect a closure, it could attempt to keep adding quads indefinitely. Also, if a closure/spot is missed, paving may not terminate properly.
- **Framework conformance:** It is a direct advancing-front method for quads. Global rule: create an all-quad mesh. Local rule: extend front by quad strips. This fits the framework as a variant of the advancing-front class.
- **Typical deviations:** Paving may incorporate specialized seam-handling (topological operations) that are not purely local geometric rules. These are custom operations to glue fronts. This is a deviation in that it uses non-geometric fixes when fronts meet.

Plastering (All-Hexahedral 3D Meshing)

- **Algorithm name:** Plastering
- **Detail name:** Plastering: All-Hexahedral Advancing-Front Filling Method
- **References (BibTeX):**

```
@article{Blacker1993,
  author = {Ted D. Blacker and Ray J. Meyers},
  title = {Seams and Wedges in Plastering: A 3-D Hexahedral Mesh Generation Algorithm},
  journal = {Engineering with Computers},
  year = {1993},
  volume = {9},
  pages = {83--93},
  doi = {10.1007/BF01199047}
}
```

- **Publication date:** 1993 (progress report)
- **Resource type:** Technical report / journal article (Sandia report)
- **Era:** Early 1990s
- **Method paradigm:** 3D advancing-front (analogy of paving). The volume is meshed by “peeling layers” of hexahedra from the boundary inward.
- **Dimension:** 3D volumetric domain (with polyhedral boundary).
- **Element type:** Hexahedra (all-hex mesh).
- **Global rule:** Fill the entire volume with hexahedral blocks. The approach builds an outer layer of hex elements and then successively removes layers until the domain collapses.
- **Problem reduction:** No explicit reduction; operates on full domain. Closures at the core may use special methods.
- **Local rule:** Remove or create one layer of hexahedra at a time. Each layer conforms to the current boundary, forming a “shell” of hexahedra. Node positions are chosen to fit geometry.
- **Engine.method:** Advancing front by removal: start with boundary, generate hexahedral elements adjacent to it, remove that layer from the domain, then repeat on the new boundary. According to Blacker’s report, hexes are removed one level at a time until nothing remains ²¹. Special closure algorithms handle the final central region.
- **Engine.objective:** Fill with valid hexahedra, aligning edges where possible, and handle geometry automatically. Aim to minimize irregular nodes (wedges, seams).
- **Output:** A hexahedral mesh (possibly with some wedges or irregular pentahedra at the center if closure is not perfect). In principle, all volume elements are hex.
- **Quality metrics:** Element quality in plastering is heuristic. The algorithm tries to produce “nice” hexahedra, but complex geometry often leads to wedges or poor shapes. Typically, edges align to principal directions from boundaries.
- **Theoretical guarantees:** None. The algorithm can fail if it cannot close the mesh in the core. The report mentions special closure algorithms but these are heuristics.
- **Computational cost:** High complexity. Hex meshing is very difficult. Plastering requires careful handling of many cases. In practice it is used with interactive or semi-automatic adjustments.
- **Practical maturity:** Research-level. Plastering was a novel idea but remains challenging. It was never fully automated in all cases. It is well-known in the hexahedral meshing community but not widely used in production (due to many corner cases).
- **Dependencies:** Needs a boundary mesh (often a surface mesh of quads). Requires algorithms for “seams and wedges” to fix topology as layers are removed.
- **Complementary suggestions:** Often followed by optimization or combined with other hex-meshing techniques. For example, if core closure fails, one might fill the core with tetrahedra and later recombine.
- **Main limitations:** Very difficult to handle arbitrary topology. Sharp concavities or complex cavities can break the layer-by-layer process. The “closure” problem (filling the last cavity with hexes) is unsolved in general and can fail.

- **Failure modes:** The algorithm may get stuck if a front cannot progress (no valid hexa can be placed without collision) or if an invalid core remains. Common issues include creating wedge-shaped elements or tunnels that cannot be filled by hexes.
- **Framework conformance:** It is exactly the 3D analog of advancing front. Global rule: fill entire volume with hex. Local rule: remove layer of hex elements. It fits the framework as a direct advancing-front method in 3D.
- **Typical deviations:** The “special closure algorithms” are additional rules not present in 2D paving. These are necessary to handle the final few elements. So it deviates by introducing ad-hoc fixes (like inserting “seams” or “wedges” when direct hex placement is impossible).

Q-Morph (Triangle-to-Quadrilateral Conversion)

- **Algorithm name:** Q-Morph
- **Detail name:** Q-Morph: Triangle-to-Quadrilateral Conversion Method
- **References (BibTeX):**

```
@article{Owen1999,
  author = {Steven J. Owen and Mark L. Staten and Scott A. Canann and Sanjay Saigal},
  title = {Q-Morph: An Indirect Approach to Advancing Front Quad Meshing},
  journal = {International Journal for Numerical Methods in Engineering},
  year = {1999},
  volume = {44},
  number = {9},
  pages = {1317--1340},
  doi = {10.1002/(SICI)1097-0207(19990330)44:9<1317::AID-NME486>3.0.CO;2-J}
}
```

- **Publication date:** 1999
- **Resource type:** Journal article (mesh generation)
- **Era:** Late 1990s
- **Method paradigm:** Indirect advancing-front. Q-Morph first generates a triangular mesh, then converts triangles to quadrilaterals by merging and transforming them in a front-based order ²³. It is a combination of indirect (triangle splitting/merging) and front-driven sequencing.
- **Dimension:** 2D surface (meshes a 2D domain or surface). The original was for surface meshes (possibly curved).
- **Element type:** Quadrilaterals (all-quad mesh). Possibly a few triangles remain if necessary, but primarily quads.
- **Global rule:** Input is first triangulated, then the entire mesh is quadrangulated. The aim is to form an all-quad mesh aligned to the domain geometry.
- **Problem reduction:** Reduce quad-meshing to a triangle-meshing plus conversion step. First solve triangle meshing, then do quad conversion (indirect method).
- **Local rule:** Given a triangulation, Q-Morph repeatedly selects one or more adjacent triangles (following an advancing front ordering) and replaces them by quads. This includes merging pairs of triangles or inserting new vertices/edges to form quads. Boundary triangles may be combined in rows. Local topological swaps and vertex insertions are used to maintain a valid quad mesh ²⁴.
- **Engine.method:** Two-stage: (1) Create an initial triangular mesh (e.g. Delaunay or advancing-front triangle mesh). (2) Use a front-based process: take the current “quad front” (initially boundary) and transform triangles adjacent to the front into quads. Edges are inserted (including new vertices) and triangles are merged. The advancing front moves inward as triangles are

converted to quads ²⁵ . Finally, small sets of triangles may be merged or split to eliminate remaining triangles.

- **Engine.objective:** Maximize the number and quality of quadrilaterals. Q-Morph aims for aligned rows of quads parallel to the boundary, and to minimize irregular nodes (vertices where valence $\neq 4$) ²⁶ .
- **Output:** An almost-all-quad mesh. In principle, it can achieve 100% quads (if boundary has even segments) since it constructs them. The final mesh has quads that are well-aligned and a small, “limited” number of irregular nodes ²⁶ .
- **Quality metrics:** Focus on alignment and limited irregularity rather than explicit angle bounds. It aims for “well-aligned rows of quadrilaterals parallel to the boundary” ²⁶ , which implies good shape for interior quads. Post-process smoothing is used to improve element shape.
- **Theoretical guarantees:** No formal guarantee on element quality or existence of solution. It guarantees an all-quad mesh if the number of boundary edges is even (similar to paving), but if not, some triangles may remain or extra measures are used.
- **Computational cost:** Medium. The operations are local and linear in the number of elements, but Q-Morph has significant topological steps (inserting nodes, merging polygons). It is more expensive than a simple direct method but still polynomial-time.
- **Practical maturity:** Well-known in academic circles. It was implemented in the LaGriT library at Sandia and others. It’s a prototype method—used mainly for demonstration and specialized meshing needs.
- **Dependencies:** Requires an initial triangle mesh (which may come from any method). The conversion step uses local mesh operations (swap, insertion), so it needs robust data structures for mesh editing.
- **Complementary suggestions:** Often followed by smoothing (as Q-Morph does). One can apply a global optimization or Laplacian smoothing after to improve quad shapes. Also, ensuring the initial triangle mesh is high quality helps the conversion.
- **Main limitations:** The method is heuristic and may produce poor quads if initial triangles are bad. It can also introduce new vertices (for boundary recovery, etc), so the node count increases. Irregular nodes are kept “limited” , but some are unavoidable.
- **Failure modes:** If the front strategy is badly chosen, triangles might not combine neatly, leaving holes or elongated quads. The paper notes it avoids expensive intersection checks, but incorrect merging order could create invalid elements that need fixing.
- **Framework conformance:** It is partially a deviation: while it uses an advancing-front order, it’s applied to an existing mesh. The “global rule” is indirect (operate on triangle mesh). One may say Problem reduction = triangulation, Local rule = triangle merging. It does not produce quads from scratch, but converts a triangulation. In the schema, it blends the indirect conversion paradigm with a front sequence.

Quadtree/Octree Subdivision

- **Algorithm name:** Quadtree/Octree Subdivision
- **Detail name:** Quadtree/Octree-Based Mesh Subdivision Method
- **References (BibTeX):**

```
@article{Yerry1983,  
  author = {Mark A. Yerry and Mark S. Shephard},  
  title = {A Modified Quadtree Approach to Finite Element Mesh Generation},  
  journal = {IEEE Computer Graphics and Applications},  
  year = {1983},  
  volume = {3},  
  number = {1},
```

```
pages = {39--46},  
doi = {10.1109/MCG.1983.262997}  
}
```

- **Publication date:** 1983
- **Resource type:** Journal article (computer graphics/meshing)
- **Era:** Early 1980s
- **Method paradigm:** Recursive subdivision (grid overlay). The domain is overlaid with a hierarchical grid (quadtree in 2D, octree in 3D), which guides mesh creation ²⁷.
- **Dimension:** 2D (quadtree) and 3D (octree). The original method was 2D, extended to 3D by Shephard et al.
- **Element type:** Typically triangles (each leaf cell is triangulated). Some variants allow quadrilaterals by merging cell halves. In 3D, cells are tetrahedra or hexahedra.
- **Global rule:** The mesh cells align with the quadtree/octree. The refinement of cells is based on geometry and a size function. The aim is to produce a graded mesh that conforms to the domain boundary.
- **Problem reduction:** The domain is effectively “reduced” by overlaying a uniform grid; large regions are subdivided to satisfy size. Then only leaf cells intersecting the geometry are used, drastically reducing complexity.
- **Local rule:** Start with a big square (or cube) covering the domain. Recursively subdivide cells: each cell is split if it intersects the geometry or if it needs to be smaller (based on size field). After subdivision, “balance” the tree so adjacent cells differ by at most one level. Finally, warp/move the grid nodes to better align with the boundary. Each leaf cell is then meshed (e.g. by splitting into triangles).
- **Engine.method:** Typical steps (Berger-Oliger style): 1) Build quadtree guided by boundary and spacing. 2) Balance tree so no hanging nodes excessively. 3) Project vertices of quadtree onto boundaries (“warp”). 4) Triangulate each leaf cell (cell clipped by boundary) individually ²⁸. In 3D, similar with octree.
- **Engine.objective:** Achieve a mesh that is graded (cells get smaller near boundaries or features) and relatively uniform elsewhere. The warping step improves element shapes by removing very short edges ²⁹.
- **Output:** A mixed mesh (triangles/quads) where the underlying structure is grid-like. The interior of large cells has a structured pattern, but cut cells may create non-uniform shapes. Overall it’s an unstructured mesh conforming to geometry.
- **Quality metrics:** Grid-based meshes naturally have good interior elements (square or cube shapes). Warping aims to avoid small angles. However, near boundaries, elements can be skewed. The text notes “elements near domain boundary are worse” ³⁰. A known quality measure is “no element much smaller or larger than neighbors” because of balanced grid.
- **Theoretical guarantees:** If refinement rules are set, it always terminates since quadtree depth is finite. Bern et al. provided variants with guarantees on minimum angle (by sufficiently refining) ³¹, but classic quadtree has no strict angle guarantee aside from warping.
- **Computational cost:** Low (fast and embarrassingly parallel). Building a quadtree is simple; triangulating each cell is local. Complexity is linear in number of cells plus a log factor for tree operations, often very efficient.
- **Practical maturity:** Classic method, widely used in practice (especially in CFD where grid alignment is acceptable). Also forms the basis of constructive octree meshing in many libraries.
- **Dependencies:** Requires the ability to intersect cells with the geometry (to decide refinements) and a size field or boundary representation. Also requires re-triangulating clipped cells.
- **Complementary suggestions:** Usually followed by smoothing or a Delaunay refinement step to improve quality and remove some of the Cartesian bias. One can also merge adjacent cells into quadrilaterals to reduce element count.

- **Main limitations:** Most elements end up aligned to a few grid directions (usually axis-aligned), which can bias simulations. Anisotropic features not aligned with the grid are hard to capture. Quality near boundaries may be poor unless extreme refinement (which increases cost).
- **Failure modes:** If geometric features are thin or diagonal, the quadtree may require excessively deep refinement or produce slivers. Without warping, element skew can be very high. Balancing can also create more cells than needed.
- **Framework conformance:** It fits the “global grid” paradigm: overlay uniform structure (global rule) and then locally refine (local rule). The local rule is recursive cell splitting. It conforms to the data-structure-driven mesh category.
- **Typical deviations:** None significant. The warping step (moving grid nodes to boundary) is an added measure not in basic tree meshing – but it is a practical enhancement and fits as a final local adjustment.

Ruppert’ s Refinement (Delaunay Refinement)

- **Algorithm name:** Ruppert’ s Refinement (Delaunay Refinement)
- **Detail name:** Ruppert’ s Delaunay Refinement Algorithm for Quality 2D Mesh Generation
- **References (BibTeX):**

```
@article{Ruppert1995,
  author = {Jim Ruppert},
  title = {A Delaunay Refinement Algorithm for Quality 2-Dimensional Mesh Generation},
  journal = {Journal of Algorithms},
  year = {1995},
  volume = {18},
  number = {3},
  pages = {548--585},
  doi = {10.1006/jagm.1995.1021}
}
```

- **Publication date:** 1995
- **Resource type:** Journal article (algorithmic analysis)
- **Era:** Mid-1990s
- **Method paradigm:** Delaunay refinement (like Chew’ s but with a stricter guarantee). Ruppert’ s algorithm incrementally refines a Delaunay triangulation to achieve a user-specified minimum angle threshold.
- **Dimension:** 2D planar (with segments/constrained input).
- **Element type:** Triangles.
- **Global rule:** Constrain the mesh so that no triangle has a circumradius/shortest-edge ratio above a threshold (equivalently, no angle below some bound).
- **Problem reduction:** Start with a triangulation of input vertices and constrained segments. Then apply refinement.
- **Local rule:** Repeatedly find the worst triangle (or segment encroachment). If a triangle is poor (e.g. its circumradius/edge > threshold), insert its circumcenter. If a boundary segment is “encroached” (circumcircle contains it), insert its midpoint. Update Delaunay.
- **Engine.method:** Similar to Chew’ s refinement, but with a clear quality threshold. In each step, do Delaunay insertion of the chosen Steiner point (circumcenter or midpoint) and retriangulate locally.
- **Engine.objective:** Achieve the minimum angle bound (or maximum radius-edge ratio). The user specifies a minimum angle (e.g. 20°), and the algorithm enforces it.

- **Output:** A conforming Delaunay triangulation meeting the quality constraint. Usually near-uniform small triangles where needed, larger elsewhere.
- **Quality metrics:** Minimum angle guarantee. Ruppert proved that for non-acute input, the algorithm terminates if threshold $< 20.7^\circ$, guaranteeing no angle smaller than that ³². In practice many implementations allow up to $\sim 30^\circ$ or more with tweaks.
- **Theoretical guarantees:** Termination and quality guarantee (minimum angle $\sim 20.7^\circ$) under mild input conditions ³². The output size is also proven to be reasonable (graded).
- **Computational cost:** Medium. The same remarks as Chew's algorithm apply: each insertion is local, and the number of insertions is bounded by domain area and feature size. Expected near $\mathcal{O}(n \log n)$ time.
- **Practical maturity:** Very mature. Ruppert's algorithm (and its implementation in Triangle) is a gold standard for guaranteed-quality meshing. It is used in industry and academia.
- **Dependencies:** Needs initial constrained Delaunay (or Delaunay) of input vertices. Also requires computing circumcenters and checking segment encroachment.
- **Complementary suggestions:** For speed, one can relax angle bounds or combine with other methods (mesh coarsening) after generation. Adaptive step size or mesh coarsening may be used for very large problems.
- **Main limitations:** If input angles are very small ($< 30^\circ$), the algorithm may have to insert many points or fail to meet high angle thresholds ³². Hard to extend to 3D robustly due to slivers.
- **Failure modes:** If the user sets an unattainable threshold (over $\sim 30^\circ$) on a complicated domain, Ruppert's algorithm may try to insert an infinite sequence or fail to terminate. Practical code often caps at $\sim 30^\circ$ for reliability.
- **Framework conformance:** Exactly fits the Delaunay refinement framework (same as Chew's paradigm). It has a strict global quality rule and a precise local insertion rule.
- **Typical deviations:** In practice, implementers allow relaxing the threshold near acute input angles (e.g. temporarily allowing a smaller local threshold) to ensure termination. This is a controlled deviation for practicality.

Structured Mapping / PDE-Based Mesh Generation (Winslow's Method)

- **Algorithm name:** Structured Mapping (PDE-Based Mesh Generation)
- **Detail name:** Structured Mapping / PDE-Based Mesh Generation Method
- **References (BibTeX):**

```
@article{Winslow1966,
  author = {A. M. Winslow},
  title = {Numerical Solution of the Quasilinear Poisson Equation in a Nonuniform Triangle Mesh},
  journal = {Journal of Computational Physics},
  year = {1966},
  volume = {1},
  number = {2},
  pages = {149--172},
  doi = {10.1016/0021-9991(66)90007-7}
}
```

- **Publication date:** 1966
- **Resource type:** Journal article (computational physics)
- **Era:** Mid-1960s (very early meshing method)

- **Method paradigm:** Elliptic (PDE-based) grid generation. Solve a system of PDEs (typically Laplace or Poisson equations) to create a smooth structured grid in a mapped computational domain. Often called Winslow's equations for mesh generation.
- **Dimension:** 2D (structured grid generation; 3D analogs exist but were developed later).
- **Element type:** Structured (usually quadrilaterals in 2D, hexahedra in 3D) – essentially mapping a regular mesh to the physical domain.
- **Global rule:** Boundary lines/curves of the mesh are mapped exactly to domain boundaries. Interior nodes are computed by solving elliptic PDEs so that the grid is smooth and orthogonal if possible.
- **Problem reduction:** Map from a simple computational space (e.g. unit square) to the physical geometry via a continuous map defined by the PDE.
- **Local rule:** The mesh is not constructed locally. Instead, the solution of the PDEs enforces local relationships: each interior mesh point is influenced by neighbors (through a discretized Laplacian). Typically, one solves for mesh coordinates $(x(u,v), y(u,v))$ such that certain metric conditions are satisfied.
- **Engine.method:** Solve a boundary value problem. For example, Winslow's method sets up $\nabla \cdot (P \nabla x) = 0, \nabla \cdot (P \nabla y) = 0$ in the computational domain, with boundary values fixed by the geometry. Here P may be a monitor function. The result is a smooth mapping, from which a structured mesh is derived.
- **Engine.objective:** Obtain a smooth grid that adapts to geometry and optionally to a prescribed size or quality function. By choosing PDE and metrics, one can encourage orthogonality or spacing.
- **Output:** A structured mesh (grid) covering the domain. Nodes follow a curvilinear grid, typically forming smooth, well-shaped elements (often near orthogonal). The output is usually a full grid, not unstructured.
- **Quality metrics:** Orthogonality of grid lines, smoothness (no grid skew, no cell size jumps). We measure how much the mesh deviates from a uniform grid in computational space. The PDE ensures minimal variation in spacing.
- **Theoretical guarantees:** The elliptic solver yields a well-posed solution that is smooth (if the boundary map is well-behaved). In practice, the resulting grid has no inverted cells because elliptic smoothing avoids reversals. There is no strict angle guarantee (elements can be skewed if geometry forces it), but control of metric can ensure some fairness.
- **Computational cost:** High. Solving a PDE system (nonlinear, in Winslow's case) over a mesh requires iterative linear solves or nonlinear iterations. For fine meshes this can be expensive.
- **Practical maturity:** Very classical. Elliptic methods like Winslow's and Thompson's were among the first systematic mesh generators (1960s-70s). They are still used in applications requiring smooth structured grids (especially in CFD for boundary-layer and interior flow).
- **Dependencies:** Requires a parametrization of the boundary (the map from computational space edges to physical boundary curves). Also a choice of PDE and monitor function (size control). A numerical PDE solver (finite differences, for example) is needed.
- **Complementary suggestions:** Often started from a coarse structured mapping and then adapt refined for solution gradients (like solving a physically-based monitor PDE). The output grid can be used as is for structured methods or further improved by optimization (monitor function tuning).
- **Main limitations:** Works only for domains that are topologically simple (rectangle-like). Cannot handle arbitrary complex or multiply-connected domains without special treatment (blocks decomposition). The user must define boundary correspondences in a structured way. High cost compared to unstructured methods.
- **Failure modes:** If the monitor function P vanishes or is poorly chosen, PDE can become degenerate. Bad boundary parameterization can cause mesh folding. Non-elliptic PDE (if misused) can cause cells to invert.

- **Framework conformance:** This method is quite different from the discrete frameworks. It's a global approach (solving PDEs), so no explicit "global rule / local rule" as defined. One could say Engine.method = solve elliptic grid PDEs, Engine.objective = smooth grid formation. It doesn't produce an "unstructured mesh" but a structured mapping. It deviates by being a continuous PDE solver rather than a combinatorial algorithm.

Sources: Each attribute above is supported by the cited literature. For example, advancing-front algorithms incrementally add elements from the boundary ¹, Delaunay refinement methods guarantee minimum angles ⁷, distmesh uses a truss-spring analogy ¹¹, FreeMesh-RL employs an SAC reinforcement learning policy ¹⁵ ¹⁴, Instant Meshes optimizes isotropy and alignment via local smoothing ¹⁶, MIQ formulates a mixed-integer global optimization ²⁰, paving builds rows of quads from the boundary ³³, and plastering extends paving to 3D by removing outer layers ²¹. These and other references validate the attributes listed.

¹ Mesh generation - Wikipedia

https://en.wikipedia.org/wiki/Mesh_generation

² ³ CPAFT: A Consistent Parallel Advancing Front Technique for Unstructured Triangular/Tetrahedral Mesh Generation

<https://arxiv.org/html/2405.20618v1>

⁴ ²² ²³ ²⁴ ²⁵ ²⁶ ³³ Q-Morph: an indirect approach to advancing front quad meshing

https://pages.cs.wisc.edu/~csverma/CS899_09/qmorph.pdf

⁵ ⁶ people.eecs.berkeley.edu

<http://people.eecs.berkeley.edu/~jrs/meshpapers/ShimadaYamadaltch.pdf>

⁷ ³² Delaunay refinement - Wikipedia

https://en.wikipedia.org/wiki/Delaunay_refinement

⁸ ⁹ Lloyd's algorithm - Wikipedia

https://en.wikipedia.org/wiki/Lloyd%27s_algorithm

¹⁰ Bowyer–Watson algorithm - Wikipedia

https://en.wikipedia.org/wiki/Bowyer%E2%80%93Watson_algorithm

¹¹ ¹² ¹³ persson.berkeley.edu

<https://persson.berkeley.edu/distmesh/persson04mesh.pdf>

¹⁴ ¹⁵ Reinforcement learning for automatic quadrilateral mesh generation: a soft actor-critic approach

<https://scispace.com/pdf/reinforcement-learning-for-automatic-quadrilateral-mesh-14sfxmut.pdf>

¹⁶ ¹⁷ ¹⁸ ¹⁹ Instant Field-Aligned Meshes | RGL

<https://rgl.epfl.ch/publications/Jakob2015Instant>

²⁰ Computer Graphics Group | UniBern

<http://cgg.unibe.ch/publications/2009/>

²¹ Plastering: A new approach to automated, 3D hexahedral mesh generation. Progress report 1 – Publications – Research

<https://www.sandia.gov/research/publications/details/plastering-a-new-approach-to-automated-3d-hexahedral-mesh-generation-progre-1992-02-01/>

²⁷ ²⁸ ²⁹ ³⁰ ³¹ people.eecs.berkeley.edu

<https://people.eecs.berkeley.edu/~jrs/papers/umg.pdf>